

Introduction

Every Python developer hits this moment: you build a class, print an instance to sanity-check it, and instead of your data you get something like `<__main__.Point object at 0x7f8a1c0d5f90>`. It looks like a bug. It isn't. Python is telling you, very literally, "you haven't told me how to represent this object yet."

This shows up constantly in interviews, too — not as a trick question, but as a signal. When you're asked to design a class (a `LinkedListNode`, a `Point`, a `Graph`), an interviewer watching you casually add `__repr__` and `__eq__` reads it as "this person understands how Python actually works," not just "this person can write a for loop." Dunder methods (short for "double underscore") are the mechanism behind nearly every built-in behavior in Python: printing, comparison, length, arithmetic, iteration. Once you understand the pattern, you stop treating Python's behavior as fixed and start treating it as something you configure.

This lesson walks through the core dunder methods you'll actually use — `__repr__`, `__str__`, `__eq__`, `__len__`, the full operator-to-method mapping — plus two related tools that round out solid class design: the `@property` decorator and the difference between `classmethod` and `staticmethod`.

Problem Overview

In plain terms: when you write ordinary Python syntax against a custom object — `print(obj)`, `obj1 == obj2`, `len(obj)`, `obj1 + obj2` — Python has to decide what that syntax *means* for your class. For built-in types like `list` or `str`, Python already knows. For a class you just wrote, it doesn't, so it falls back to defaults:

- `print(obj)` shows the class name and memory address.
- `obj1 == obj2` checks identity (are they the *same object in memory*), not whether their data matches.
- `len(obj)` raises a `TypeError`, because Python has no idea what "length" should mean for your class.
- `obj1 + obj2` raises a `TypeError` too, for the same reason.

None of this is broken. Python is refusing to guess. It gives you a set of hook methods — the dunder methods — and whichever ones you define are the ones Python will call when that syntax is used on

your object.

Example

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p1 = Point(2, 3)
p2 = Point(2, 3)

print(p1)          # <__main__.Point object at 0x000001F3B2A1C4D0>
print(p1 == p2)    # False
print(len(p1))     # TypeError: object of type 'Point' has no len()
```

Walking through each line: `print(p1)` shows a memory address because Python doesn't know what a "readable version" of a `Point` looks like. `p1 == p2` is `False` even though both points hold `(2, 3)`, because Python's default `==` is really asking "is this the exact same object?" — and `p1` and `p2` are two separate objects that just happen to contain equal data. `len(p1)` fails outright because there's no default definition of length for an arbitrary object at all.

Intuition

Here's how an engineer who's seen this before thinks about it. Every one of these built-in operations — `print()`, `==`, `len()`, `+` — is really just syntax sugar. Python doesn't have special-cased logic baked in for your class; instead, when it sees `print(obj)`, it looks for a method named `__repr__` on `obj`'s class and calls that. When it sees `obj1 == obj2`, it calls `obj1.__eq__(obj2)`. When it sees `len(obj)`, it calls `obj.__len__()`.

Think of it like a doorbell: you press the button (write `print(obj)`), and a wire behind the wall (the `__repr__` method) is what actually makes the chime ring. You don't rewire how the button itself works — you just wire up what happens when it's pressed. That's the entire trick behind dunder methods. Once you see this, "operator overloading" stops being a special topic and becomes one repeated pattern: pick the operation, define the matching method, done.

Brute Force Solution

There isn't really a "brute force" alternative to dunder methods in the algorithmic sense — but there is a brute-force *habit* worth naming, because it's what people reach for before they learn this pattern:

writing manual comparison and display functions instead of hooking into Python's data model.

Idea: Write standalone helper functions like `points_equal(p1, p2)` and `print_point(p)` instead of `__eq__` and `__repr__`.

```
def points_equal(p1, p2):
    return p1.x == p2.x and p1.y == p2.y

def print_point(p):
    print(f"Point({p.x}, {p.y})")
```

Advantages: Works, requires no knowledge of dunder methods, easy to understand for a beginner.

Disadvantages: You lose native syntax — `p1 == p2` still returns `False`, `print(p1)` still shows a memory address, and every caller has to remember to use your custom function instead of the built-in operator. It doesn't compose with the rest of Python (sorting, `in` checks, debuggers, f-strings all bypass your helper). It also doesn't scale — every new class needs its own bespoke set of helper functions instead of following one consistent interface.

Complexity: $O(1)$ time and space for each individual call — the cost here isn't computational, it's the loss of language-level consistency.

Optimal Solution

The optimal approach is to implement the dunder methods directly, so the *built-in syntax itself* does what you want.

You write

```
print(obj) / repr(obj)
str(obj)
obj1 == obj2
len(obj)
obj1 + obj2
obj1 - obj2
obj1 < obj2
```

Python calls

```
obj.__repr__()
obj.__str__() (falls back to __repr__ if undefined)
obj1.__eq__(obj2)
obj.__len__()
obj1.__add__(obj2)
obj1.__sub__(obj2)
obj1.__lt__(obj2)
```

The rule of thumb: if you only have time to define one of `__repr__` or `__str__`, define `__repr__`. Python uses it as the fallback for `str()`, for f-strings, for debuggers, and for the console — it's the one that shows up almost everywhere by default. `__str__` is only worth adding separately when you want a more human-friendly version distinct from the debug-oriented one.

Beyond direct operator hooks, two more tools round out clean class design:

- **`@property`** lets an attribute look like a plain value (`point.area` , no parentheses) while it's actually computed fresh every time it's accessed — like a car's odometer, which looks like a static number on the dash but is recalculated continuously. This gives you the validation benefits of a getter/setter pair without forcing callers to write `get_area()` .
- **`classmethod` vs `staticmethod`** — a `classmethod` receives the class itself as its first argument, so it's the right tool for alternate constructors (`Point.from_tuple((2, 3))`). A `staticmethod` receives nothing extra; it's a plain function that just happens to live inside the class because it's related to it, with no access to instance or class data.

Python Code

```
class Point:
    """A 2D point with equality, length, and arithmetic support."""

    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        return f"Point({self.x}, {self.y})"

    def __eq__(self, other):
        if not isinstance(other, Point):
            return NotImplemented
        return self.x == other.x and self.y == other.y

    def __len__(self):
        # Distance from origin, truncated to an int, so len() stays well-defined.
        return int((self.x ** 2 + self.y ** 2) ** 0.5)

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    @property
    def area(self):
        # Treats the point as the corner of a square region for demo purposes.
        return abs(self.x * self.y)

    @classmethod
    def from_tuple(cls, coords):
        return cls(coords[0], coords[1])

    @staticmethod
    def distance(p1, p2):
        return ((p1.x - p2.x) ** 2 + (p1.y - p2.y) ** 2) ** 0.5
```

Code Walkthrough

- `__repr__` returns a string built from the instance's own `x` and `y` — this is what `print()`, f-strings, and the console all fall back to.
- `__eq__` first checks `isinstance(other, Point)` and returns `NotImplemented` (not `False`) if the comparison doesn't make sense — this lets Python correctly fall back to `other.__eq__(self)` or a final `False` instead of silently producing a wrong answer against an unrelated type.
- `__len__` must return a non-negative `int`; that's a hard requirement from Python's data model, which is why the raw float distance is wrapped in `int()`.
- `__add__` returns a brand-new `Point` rather than mutating `self`, matching how `+` behaves on every built-in numeric type.
- `area` is decorated with `@property`, so `p.area` reads like a plain attribute but recomputes on every access — no stale cached value to worry about.
- `from_tuple` is a `classmethod`; `cls(...)` is used instead of `Point(...)` so subclasses that inherit this constructor still build the correct subclass.
- `distance` is a `staticmethod` because it needs no instance or class state — it's just grouped here because it's conceptually related to `Point`.

Dry Run

```
p1 = Point(2, 3)
p2 = Point.from_tuple((2, 3))

print(p1)           # Point(2, 3)           -> calls __repr__
print(p1 == p2)     # True                  -> calls __eq__, compares x and y
print(len(p1))      # 3                    -> int(sqrt(2**2 + 3**2)) = int(3.60...) = 3
p3 = p1 + p2
print(p3)           # Point(4, 6)           -> calls __add__
print(p1.area)      # 6                    -> property, computed on access
print(Point.distance(p1, p2)) # 0.0         -> staticmethod, no dunder involved
```

Each line demonstrates the same underlying idea: standard syntax (`print`, `==`, `len`, `+`) is quietly being routed to a method you wrote.

Complexity Analysis

Every dunder method here does $O(1)$ work — a couple of attribute lookups and arithmetic operations — so `__repr__`, `__eq__`, `__len__`, and `__add__` are all $O(1)$ time and $O(1)$ space. This will hold for

most simple value classes; it only changes if the class wraps a collection (e.g., a custom list-like class), where `__len__` or `__eq__` would become $O(n)$ because they'd need to inspect every element.

Alternative Solutions

The `dataclasses` module is worth mentioning as a comparison point: `@dataclass` auto-generates `__init__`, `__repr__`, and `__eq__` for you based on declared fields.

```
from dataclasses import dataclass

@dataclass
class Point:
    x: int
    y: int
```

This gets you correct `__repr__` and `__eq__` behavior with zero manual code, and it's the right choice for simple data containers. It's worth reaching for `dataclasses` in real projects — but understanding what it's generating under the hood (exactly the methods covered above) is what lets you debug it, extend it with custom `__len__` or `__add__`, or know when you need the manual version instead — for instance, when equality needs custom logic beyond simple field comparison.

Edge Cases

- **Comparing against the wrong type** (`p1 == "not a point"`): handled by the `isinstance` check in `__eq__`, returning `NotImplemented` instead of crashing.
- **`__len__` returning a negative number**: Python's data model explicitly forbids this and will raise a `ValueError` — always ensure the computed value can't go negative.
- **Only defining `__eq__` without `__hash__`**: Python automatically sets `__hash__` to `None` once you define `__eq__`, making instances unhashable and unusable as dict keys or in sets — define `__hash__` explicitly if you need both.
- **Origin point** (`Point(0, 0)`): `len()` correctly returns `0`, and `area` correctly returns `0` — good default values to test against.
- **`__add__` with a non-`Point` argument**: the current implementation will raise an `AttributeError` on `other.x` — a production version should validate `other`'s type the same way `__eq__` does.

Common Mistakes

1. Defining `__str__` but not `__repr__` — since `__repr__` is the fallback everywhere `__str__` isn't explicitly used (debuggers, containers, f-string defaults in some contexts), skipping it leaves gaps.
2. Returning `False` instead of `NotImplemented` from `__eq__` when the types don't match, which silently breaks Python's ability to try the reverse comparison.
3. Mutating `self` inside `__add__` instead of returning a new object — this breaks the expectation that `a + b` doesn't change `a`.
4. Forgetting that defining `__eq__` disables the default `__hash__`, then being confused when instances can't go in a `set`.
5. Confusing `classmethod` and `staticmethod` — assuming `staticmethod` needs access to the instance, when the entire point of `staticmethod` is that it doesn't.
6. Putting expensive computation behind a `@property` without realizing it re-runs on every single access — if the calculation is costly, cache it explicitly instead of assuming `@property` memoizes for you.

Interview Questions

- What's the difference between `__repr__` and `__str__`, and which one does Python fall back to?
- Why does `__eq__` return `NotImplemented` instead of `False` for an incompatible type?
- What happens to `__hash__` when you define `__eq__`, and why does that matter for using an object as a dict key?
- Walk through what Python actually does when it evaluates `obj1 + obj2`.
- When would you use a `classmethod` instead of a plain `__init__` overload?
- Give an example where a `staticmethod` is a better fit than a `classmethod` or instance method.

Similar Problems

- **Design a HashMap / Design a HashSet** (LeetCode) — relies on the same idea of defining custom `__eq__` and `__hash__` for objects used as keys.
- **LRU Cache** — often implemented with a custom class where `__repr__` and comparison methods make debugging the internal linked list dramatically easier.
- **Min Stack / Design classes in general** — any "design a data structure" interview question benefits from the same dunder-method discipline for clean, debuggable output.

Key Takeaways

Python's default object behavior — a memory address on print, identity-based equality, no length — isn't a limitation, it's an invitation. Dunder methods are the wiring behind ordinary syntax: `print()` calls `__repr__`, `==` calls `__eq__`, `len()` calls `__len__`, `+` calls `__add__`, and so on, consistently, across every operator Python supports. Once that pattern clicks, operator overloading stops being "magic" and becomes a simple decision: pick the syntax you want to support, define the matching method. `@property` extends the same philosophy to attributes — compute freely, but keep the plain-dot syntax callers expect. And `classmethod` versus `staticmethod` is really just "does this function need the class or not" — nothing more mysterious than that.

Watch the Video

For the full walkthrough with live code and the doorbell and odometer analogies in action, watch the video here: {LINK}

About the Series

This lesson is part of **Fun with Learning Technology**, a series that takes the weird, easy-to-misunderstand corners of code and makes them click — one concept, one clear mental model at a time.

Call To Action

If this helped clear up how Python's data model actually works, drop a like on the video and subscribe on YouTube for the rest of the series. For deeper written breakdowns like this one, subscribe to the Substack. And if something felt unclear or you want a specific topic covered next, leave a comment — it directly shapes what gets made next.

Quick Review

`__repr__` vs `__str__`: when does Python call `__repr__`?

Fallback when `__str__` is missing, plus always in REPL/debugger echo, containers, and `print()` if `__str__` undefined.

How does `==` work without `__eq__` defined?

Falls back to identity comparison (`is`) — same object in memory, not equal values.

What must `len()` call to work on a custom object?

`__len__`, which must return a non-negative int; `TypeError` if missing.

`obj1 + obj2` triggers which dunder, and the tricky part?

`__add__(self, other)`; must return `NotImplemented` (not raise) if other's type is unsupported, so Python can try `__radd__`.

`@property` vs `@staticmethod` vs `@classmethod` — when to use each?

`@property`: computed attribute accessed without `()`; `@staticmethod`: no `self/cls`, utility logic;
`@classmethod`: takes `cls`, builds/modifies class-level state.

Common bug when overloading operators like `__eq__`?

Forgetting to also define `__hash__` — objects become unhashable, breaking use in sets/dicts.

Python Lesson 21: OOP: Dunder Methods and Properties (`__repr__`, `__eq__`, `__len__`, operator overloading, `@property`, and `class/static methods`) — free Python cheat sheet